

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 3. Python Data Types

Lecture: 22. Literals or Constants

Section 1: Lecture Summary

This lecture thoroughly explores the concept of **literals** in Python, explaining what literals are and their importance as direct constant values used in programs. It starts by differentiating variables that are assigned direct values (literals) from those assigned results of expressions or input. The lecture then dives into the various **types of literals**: integer, float, boolean, complex, and string literals. New important syntax features such as the use of underscores (_) for readability in numeric literals are introduced and demonstrated with examples, including valid and invalid usages. The lecture also clarifies common errors linked to literals, like syntax and name errors. Finally, it explains string literals in single, double, and triple quotes and highlights subtle distinctions in usage. Throughout, the concepts are reinforced with practical demonstrations in Jupyter Notebook.

Section 2: Key Concepts and Explanations

- What are Literals?

- Literals are **direct constant values** explicitly written in code.
- Variables may hold literals, results of computations, or input data.
- Literals are synonymous with constants.
- Example: `a = 10`` (10 is literal), vs `c = a + b`` (c receives a computed value, not a literal).

- Types of Literals

1. Integer Literals

- Direct integer values like ``201``.
- Can be very large numbers.
- Cannot include commas for digit grouping in code; instead, use **underscores (`_`) as digit separators** for readability (e.g., ``13_52_46_198``).
- Underscores must only be between digits, not at the start or end.
- Examples of invalid uses: ``358_``, ``_125``.

2. Float Literals

- Numbers with decimals like ``9.75``, or in scientific notation like ``12E2`` or ``12.5e-2``.
- Can also use underscores between digits and decimals for readability (e.g., ``125.67`` or ``12_5.6_7``).
- Underscores cannot be placed directly before or after a decimal point.
- Checking type with ``type()`` demonstrates literals with decimal points are floats.

3. Boolean Literals

- Only two values: ``True`` and ``False``.
- Must be capitalized; all lowercase ``true`` or ``false`` is invalid and raises errors.
- Integers like ``1`` are not boolean literals.

4. Complex Literals

- Complex numbers with real and imaginary parts, e.g., ``4+5j``, ``12+14j``.
- Both real and imaginary parts can be integers or floats (e.g., ``1.4+2.5j``).
- Underscores can be used within parts for readability.
- The letter ``j`` denotes the imaginary unit.

5. String Literals

- Represent text, such as words or sentences.

- Can be enclosed in single quotes `'Alexa'`, double quotes `"Alexa"`, or triple quotes `'''Alexa'''`.

- Double and single quotes can be nested: e.g., `'''Alexa'''` produces a string including double quotes inside.

- Triple quotes allow multi-line strings or strings containing both single and double quotes.

- Common Errors with Literals

- `SyntaxError` when literals are malformed (e.g., trailing underscores).

- `NameError` when incorrect capitalization is used for booleans (`false`` instead of `False``).

- Understanding error messages helps in debugging.

Section 3: Example Code and Use Cases

Integer literals with underscores for readability

Invalid integer literals examples (will cause SyntaxError or NameError)

`c = 358_` `SyntaxError: invalid decimal literal`

`d = _125` `NameError: name '_125' is not defined`

Float literals examples

Scientific notation float literals

Float literals with underscores

Boolean literals (correct usage)

Boolean literals (incorrect usage - Uncommenting causes error)

`c = false` `NameError: name 'false' is not defined`

Complex literals with underscores

Complex literals with floats and underscores

String literals

```
a = 201
b = 13_52_46_198
print(a, b)   # Output: 201 135246198

a = 9.75
b = 15.0
print(a, b)   # Output: 9.75 15.0
print(type(b))   # Output: <class 'float'>

c = 12E2    # Equivalent to 12*10^2 = 1200.0
d = 12.5e-2   # 0.125
print(c, d)   # Output: 1200.0 0.125

e = 125.67
f = 12_5.6_7
print(e, f)   # Output: 125.67 125.67

a = True
b = False
print(a, b)   # Output: True False
```

```
a = 4 + 5j
b = 12 + 14j
print(a, b) # Output: (4+5j) (12+14j)

c = 1.4 + 2.5j
d = 1_2 + 1_4j
print(c, d) # Output: (1.4+2.5j) (12+14j)

name = 'Alexa'
name1 = "Alexa"
name2 = '"Alexa"'
print(name, name1, name2) # Output: Alexa Alexa "Alexa"
```

Section 4: Key Takeaways

- Literals are **direct constant values** hardcoded in a program and can be integers, floats, booleans, complex numbers, or strings.
- Use **underscores** `\_` within numeric literals (integers and floats) to improve readability but never at the start or end.
- Float literals support decimal points and scientific notation (`E` or `e`).
- Boolean literals are only **`True`** and **`False`** (case-sensitive).
- Complex literals combine real and imaginary parts with a `j` suffix; both parts can be floats or integers.
- Strings can be enclosed in single, double, or triple quotes; each style has specific use cases especially regarding embedded quote characters.
- Syntax and name errors are common pitfalls with literals; understanding error messages helps in correcting them.
- Practicing literals and their correct syntax in environments like Jupyter Notebook reinforces understanding and error detection.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.



Literals

What are Literals ?

Literals

What are Literals ?

1. Integer

Literals

What are Literals ?

1. Integer
2. Float

Literals

What are Literals ?

1. Integer
2. Float
3. Boolean

Literals

What are Literals ?

1. Integer
2. Float
3. Boolean
4. Complex

Literals

What are Literals ?

1. Integer
2. Float
3. Boolean
4. Complex
5. String

Literals

What are Literals ?

a = 10

Literals

What are Literals ?

a = 10

b = 5

Literals

What are Literals ?

a = 10

b = 5

c = a + b

Literals

What are Literals ?

a = 10

b = 5

c = a + b

d = input('Enter name')

Literals

What are Literals ?

a = 10

constants

b = 5

c = a + b

d = input('Enter name')

Literals

integer Literals

Literals

integer Literals

a = 201

Literals

integer Literals

a = 201

b = 135246198

Literals

integer Literals

a = 201

b = 135246_198



Literals

integer Literals

a = 201

b = 135_246_198



Literals

integer Literals

a = 201

b = 1352_46_198

Literals

integer Literals

a = 201

b = 13_52_46_198

Literals

integer Literals

a = 201

b = 13_52_46_198

c = 358_

Literals

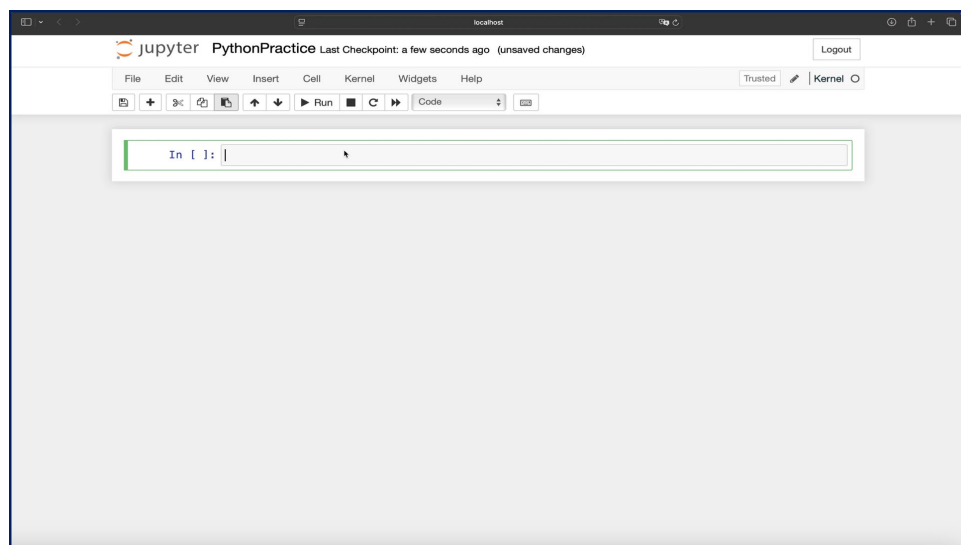
integer Literals

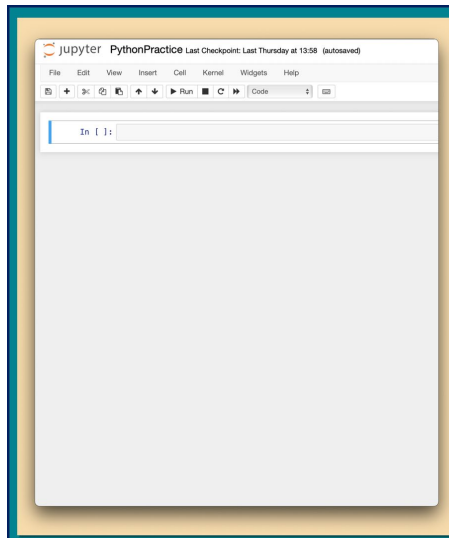
a = 201

b = 13_52_46_198

c = 358_ ❌

d = _125





Literals

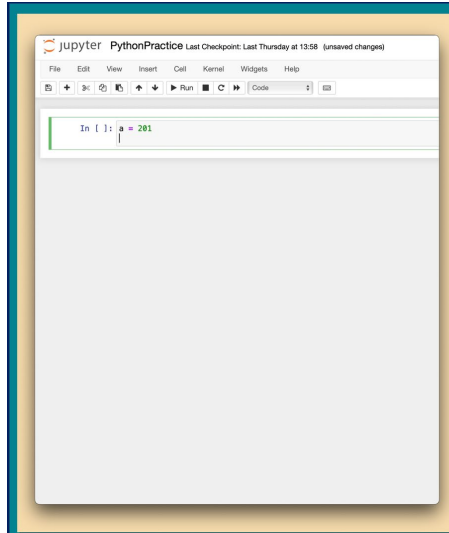
integer Literals

a = 201

b = 13_52_46_198

c = 358_ ❌

d = _125 ❌



The image shows a Jupyter Notebook window titled 'PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The interface includes a menu bar with 'File', 'Edit', 'View', 'Insert', 'Cell', 'Kernel', 'Widgets', and 'Help'. Below the menu is a toolbar with icons for running, saving, and other actions. The main area contains a single code cell with the text 'In []: a = 201'.

Literals

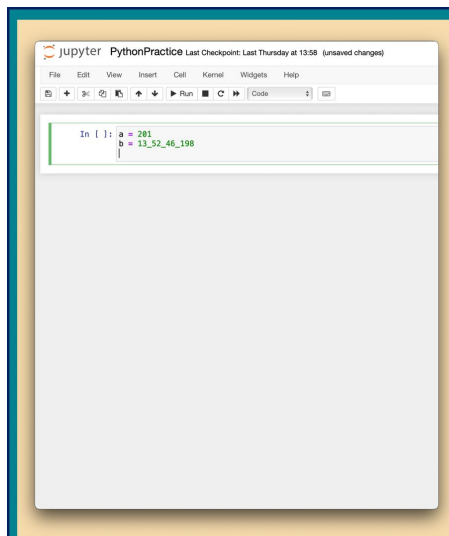
integer Literals

a = 201

b = 13_52_46_198

c = 358_ ❌

d = _125 ❌



The image shows a Jupyter Notebook window titled 'PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The interface includes a menu bar with 'File', 'Edit', 'View', 'Insert', 'Cell', 'Kernel', 'Widgets', and 'Help'. Below the menu is a toolbar with icons for running, saving, and other actions. The main area contains a single code cell with two lines of text: 'In []: a = 201' and 'b = 13_52_46_198'.

Literals

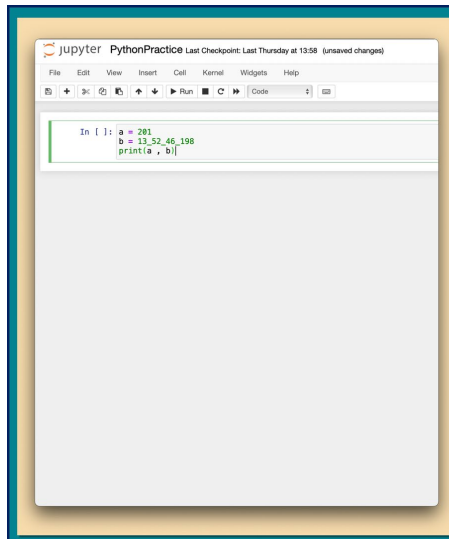
integer Literals

a = 201

b = 13_52_46_198

c = 358_ ❌

d = _125 ❌



A screenshot of a Jupyter Notebook interface. The title bar shows 'jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar contains icons for saving, undo, redo, running, and other actions. The code cell contains the following Python code:

```
In [ ]: a = 201
        b = 13_52_46_198
        print(a, b)
```

Literals

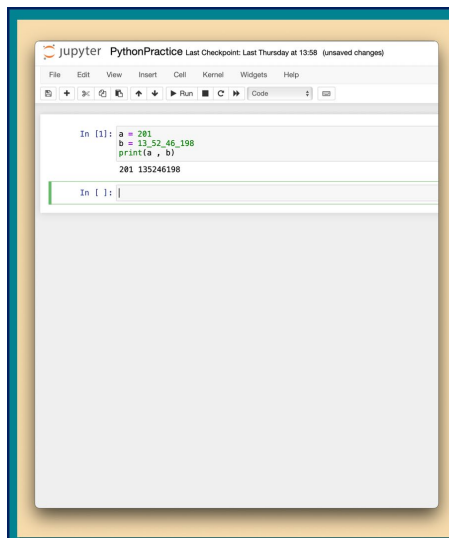
integer Literals

a = 201

b = 13_52_46_198

c = 358_ ❌

d = _125 ❌



A screenshot of a Jupyter Notebook interface showing the output of the code cell. The title bar and menu bar are the same as in the previous image. The code cell contains the same Python code. The output cell below it shows the result of the print statement:

```
201 135246198
```

Literals

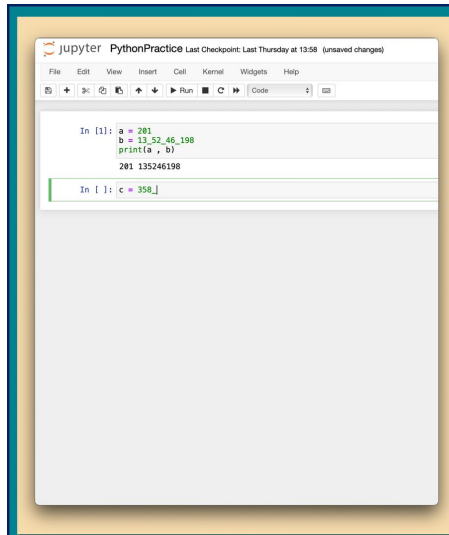
integer Literals

a = 201

b = 13_52_46_198

c = 358_ ❌

d = _125 ❌



A screenshot of a Jupyter Notebook interface. The top bar shows 'jupyter PythonPractice' and 'Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar has icons for running, saving, and other actions. The code area shows two input cells. The first cell contains:

```
In [1]: a = 201  
b = 13_52_46_198  
print(a, b)
```

The output of this cell is '201 135246198'. The second cell contains:

```
In [ ]: c = 358_
```

Literals

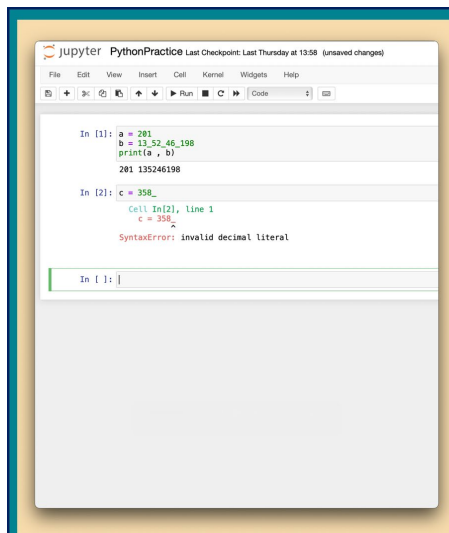
integer Literals

a = 201

b = 13_52_46_198

c = 358_ ❌

d = _125 ❌



A screenshot of a Jupyter Notebook interface, similar to the one above. The code area shows two input cells. The first cell is identical to the one in the first image. The second cell contains:

```
In [2]: c = 358_  
        Cell In[2], line 1  
        c = 358_  
        ^  
SyntaxError: invalid decimal literal
```

Literals

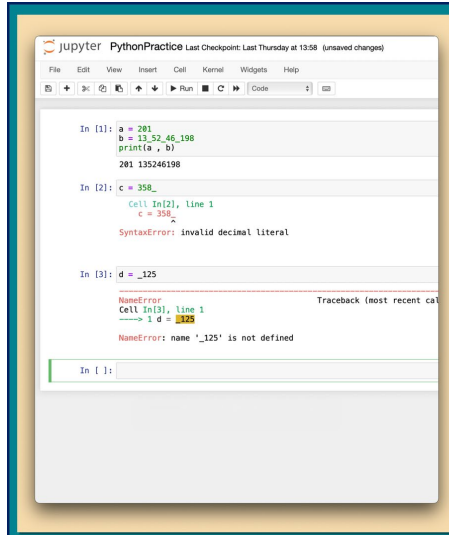
integer Literals

a = 201

b = 13_52_46_198

c = 358_ ❌

d = _125 ❌



A screenshot of a Jupyter Notebook interface. The top bar shows 'jupyter PythonPractice' and a timestamp. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar has icons for running, saving, and other actions. The code area shows three input cells. The first cell contains code that prints the value of 'a' (201) and 'b' (13_52_46_198). The second cell contains code that assigns 'c' to 358_ and results in a 'SyntaxError: invalid decimal literal'. The third cell contains code that assigns 'd' to _125 and results in a 'NameError: name '_125' is not defined'. The output area shows the results of the first cell and the error messages for the second and third cells.

```
In [1]: a = 201
        b = 13_52_46_198
        print(a, b)
        201 135246198

In [2]: c = 358_
        Cell In[2], line 1
        c = 358_
        SyntaxError: invalid decimal literal

In [3]: d = _125
        NameError                                Traceback (most recent call last)
        Cell In[3], line 1
        ----> 1 d = _125
        NameError: name '_125' is not defined

In [ ]:
```

Literals

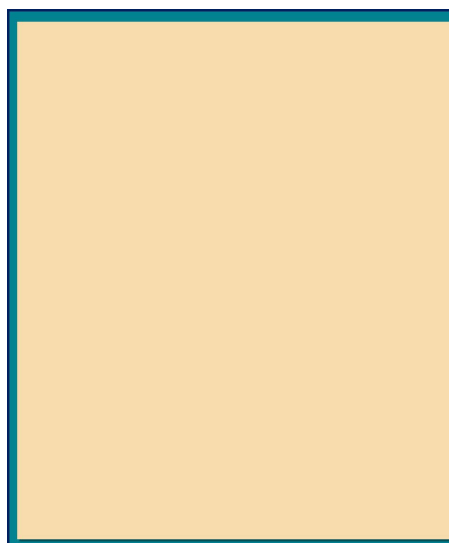
integer Literals

a = 201

b = 13_52_46_198

c = 358_ ❌

d = _125 ❌



Literals

Literals
float Literals

Literals
float Literals
a = 9.75

Literals

float Literals

a = 9.75

b = 15

Literals

float Literals

a = 9.75

b = 15.0

Literals

float Literals

a = 9.75

b = 15.0

c = 12E2

Literals

float Literals

a = 9.75

b = 15.0

c = 12E2

d = 12.5e-2

Literals

float Literals

a = 9.75

b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

Literals

float Literals

a = 9.75

b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 125.67

Literals

float Literals

a = 9.75

b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.67

Literals

float Literals

a = 9.75

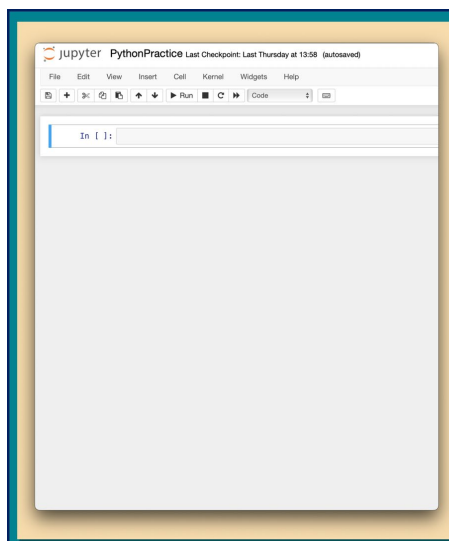
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9



Literals

float Literals

a = 9.75

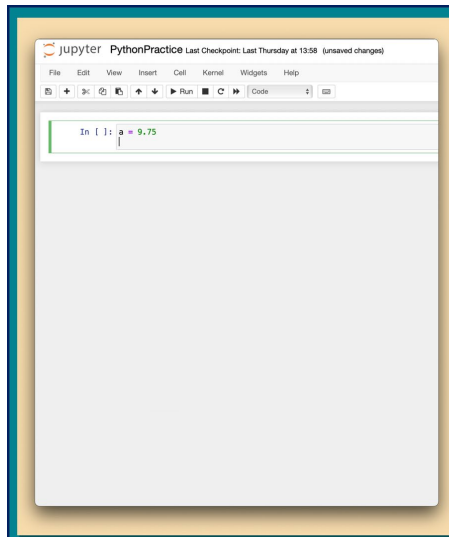
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌

A screenshot of a Jupyter Notebook interface. The title bar shows 'jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar contains icons for adding, deleting, and running cells, as well as a code editor icon. The code area shows a single cell with the text 'In []: a = 9.75'.

Literals

float Literals

a = 9.75

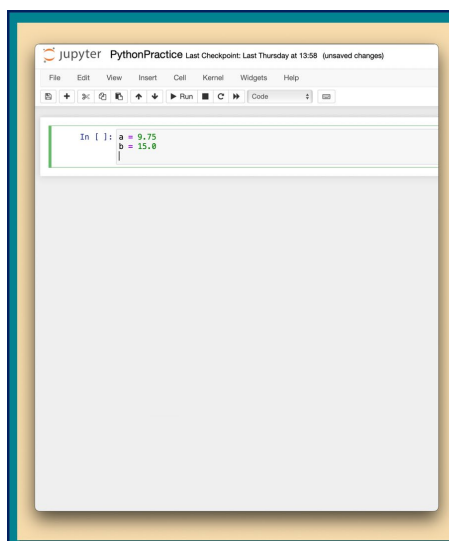
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌

A screenshot of a Jupyter Notebook interface, similar to the one above. The title bar shows 'jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The menu bar and toolbar are the same. The code area shows a single cell with two lines of code: 'a = 9.75' and 'b = 15.0'.

Literals

float Literals

a = 9.75

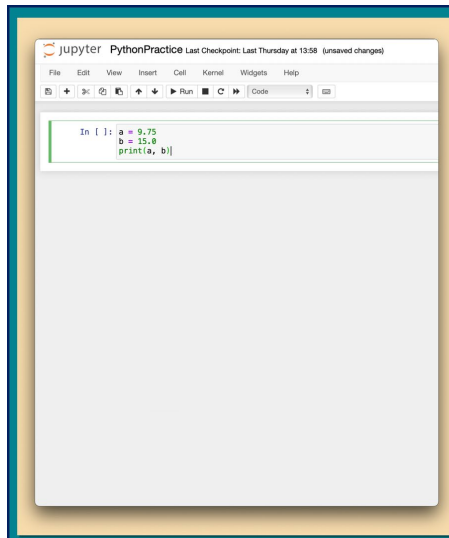
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌

A screenshot of a Jupyter Notebook interface. The title bar shows 'jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar contains icons for running, saving, and other actions. The code editor shows a Python cell with the following code:

```
In [ ]: a = 9.75  
        b = 15.0  
        print(a, b)
```

Literals

float Literals

a = 9.75

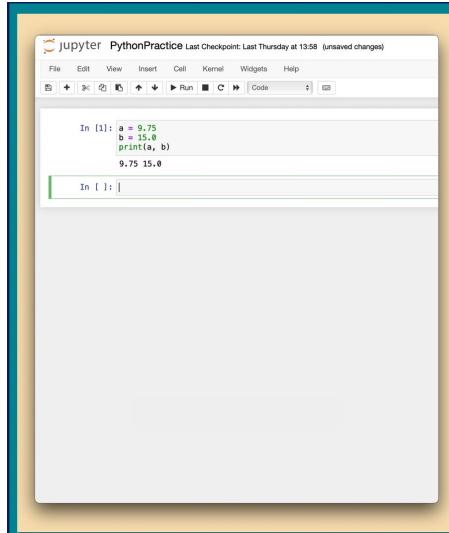
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌



```
jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
In [1]: a = 9.75
        b = 15.0
        print(a, b)
        9.75 15.0
In [ ]:
```

Literals

float Literals

a = 9.75

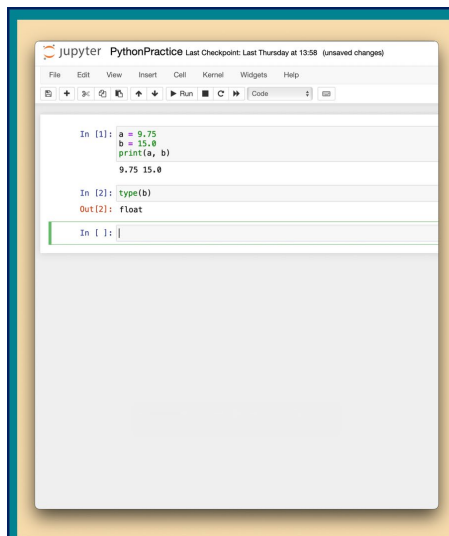
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌



```
jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
In [1]: a = 9.75
        b = 15.0
        print(a, b)
        9.75 15.0
In [2]: type(b)
Out[2]: float
In [ ]:
```

Literals

float Literals

a = 9.75

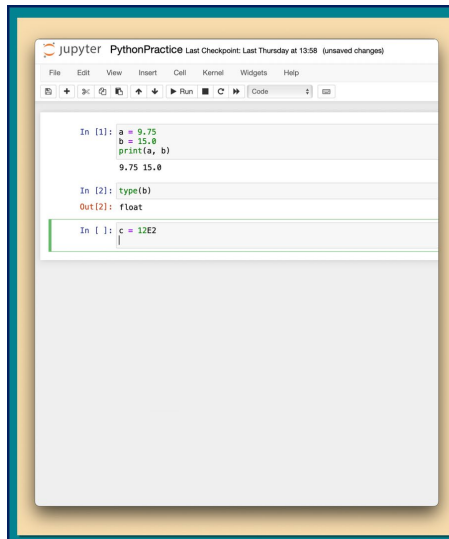
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌



A Jupyter Notebook interface with the title 'PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, running, and code execution. The notebook contains three input cells:

```
In [1]: a = 9.75  
        b = 15.0  
        print(a, b)  
9.75 15.0
```

```
In [2]: type(b)  
Out[2]: float
```

```
In [ ]: c = 12E2
```

Literals

float Literals

a = 9.75

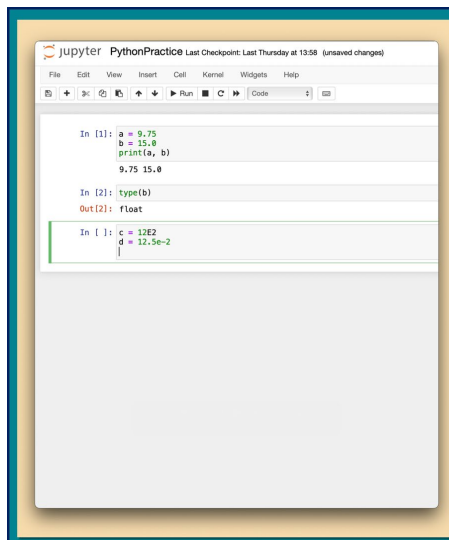
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌



A Jupyter Notebook interface with the title 'PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, running, and code execution. The notebook contains three input cells:

```
In [1]: a = 9.75  
        b = 15.0  
        print(a, b)  
9.75 15.0
```

```
In [2]: type(b)  
Out[2]: float
```

```
In [ ]: c = 12E2  
        d = 12.5e-2
```

Literals

float Literals

a = 9.75

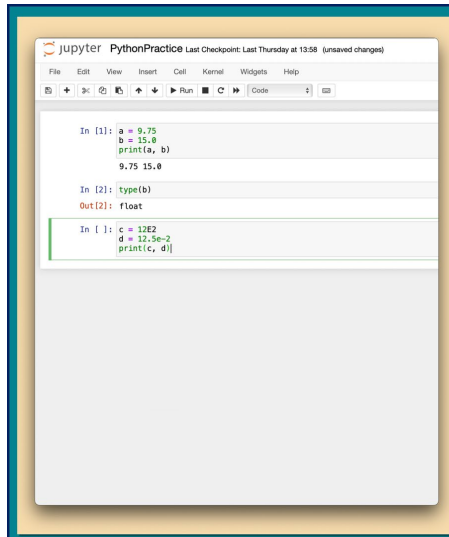
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌



A Jupyter Notebook interface with the title 'PythonPractice Last Checkpoint: Last Thursday at 13:58 (unsaved changes)'. The notebook contains three input cells. The first cell has code: `a = 9.75`, `b = 15.0`, and `print(a, b)`, with output `9.75 15.0`. The second cell has code: `type(b)`, with output `float`. The third cell has code: `c = 12E2`, `d = 12.5e-2`, and `print(c, d)`, with output `1200.0 0.125`.

Literals

float Literals

`a = 9.75`

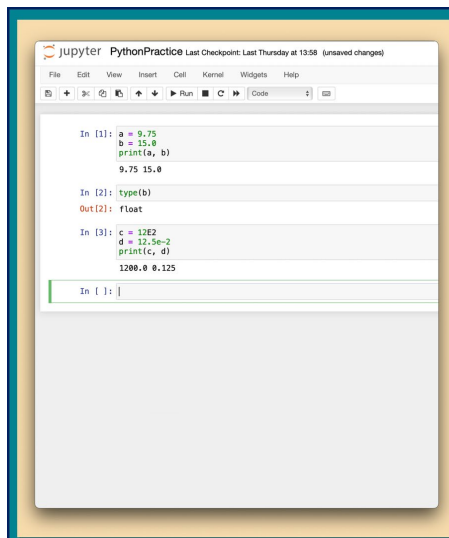
`b = 15.0`

`c = 12E2`

`d = 12.5e-2` (0.125)

`e = 12_5.6_7`

`f = 5_._9` ❌



A Jupyter Notebook interface with the title 'PythonPractice Last Checkpoint: Last Thursday at 13:58 (unsaved changes)'. The notebook contains three input cells. The first cell has code: `a = 9.75`, `b = 15.0`, and `print(a, b)`, with output `9.75 15.0`. The second cell has code: `type(b)`, with output `float`. The third cell has code: `c = 12E2`, `d = 12.5e-2`, and `print(c, d)`, with output `1200.0 0.125`.

Literals

float Literals

`a = 9.75`

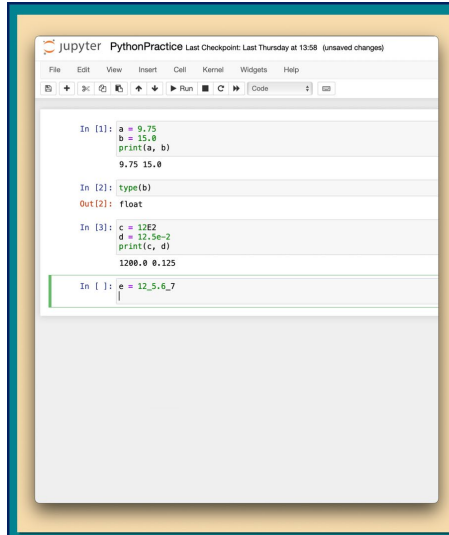
`b = 15.0`

`c = 12E2`

`d = 12.5e-2` (0.125)

`e = 12_5.6_7`

`f = 5_._9` ❌



A Jupyter Notebook interface with the title 'PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The notebook contains three input cells. The first cell (In [1]) contains the code `a = 9.75`, `b = 15.0`, and `print(a, b)`, with output `9.75 15.0`. The second cell (In [2]) contains `type(b)`, with output `Out[2]: float`. The third cell (In [3]) contains `c = 12E2`, `d = 12.5e-2`, and `print(c, d)`, with output `1200.0 0.125`. A fourth input cell (In [4]) contains the code `e = 12_5.6_7` and is currently selected.

Literals

float Literals

a = 9.75

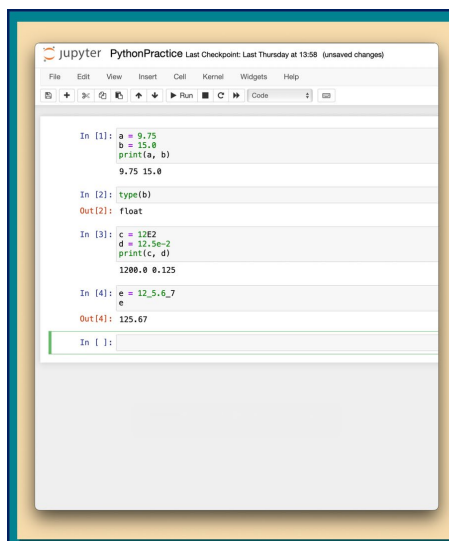
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌



A Jupyter Notebook interface with the title 'PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The notebook contains four input cells. The first cell (In [1]) contains the code `a = 9.75`, `b = 15.0`, and `print(a, b)`, with output `9.75 15.0`. The second cell (In [2]) contains `type(b)`, with output `Out[2]: float`. The third cell (In [3]) contains `c = 12E2`, `d = 12.5e-2`, and `print(c, d)`, with output `1200.0 0.125`. The fourth cell (In [4]) contains the code `e = 12_5.6_7` and `e`, with output `Out[4]: 125.67`. A fifth input cell (In [5]) is currently selected and empty.

Literals

float Literals

a = 9.75

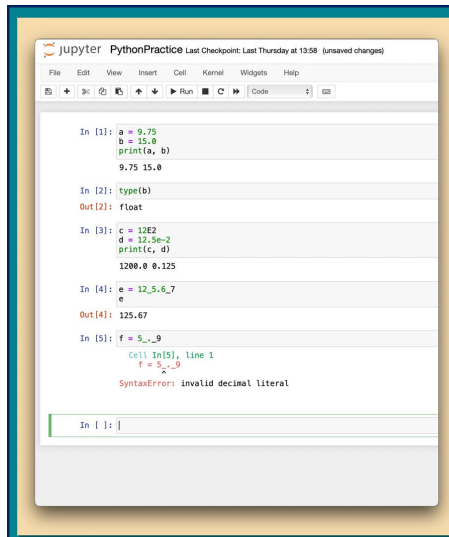
b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

e = 12_5.6_7

f = 5_._9 ❌



A screenshot of a Jupyter Notebook interface. The title bar says 'jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar has icons for saving, running, and other notebook functions. The code area shows the following:

```
In [1]: a = 9.75
        b = 15.0
        print(a, b)
9.75 15.0

In [2]: type(b)
Out[2]: float

In [3]: c = 12E2
        d = 12.5e-2
        print(c, d)
1200.0 0.125

In [4]: e = 12_5.6_7
        e
Out[4]: 125.67

In [5]: f = 5._.9
Cell In[5], line 1
f = 5._.9
      ^
SyntaxError: invalid decimal literal
```

The last line of code has a syntax error highlighted in red.

Literals

float Literals

a = 9.75

b = 15.0

c = 12E2

d = 12.5e-2 (0.125)

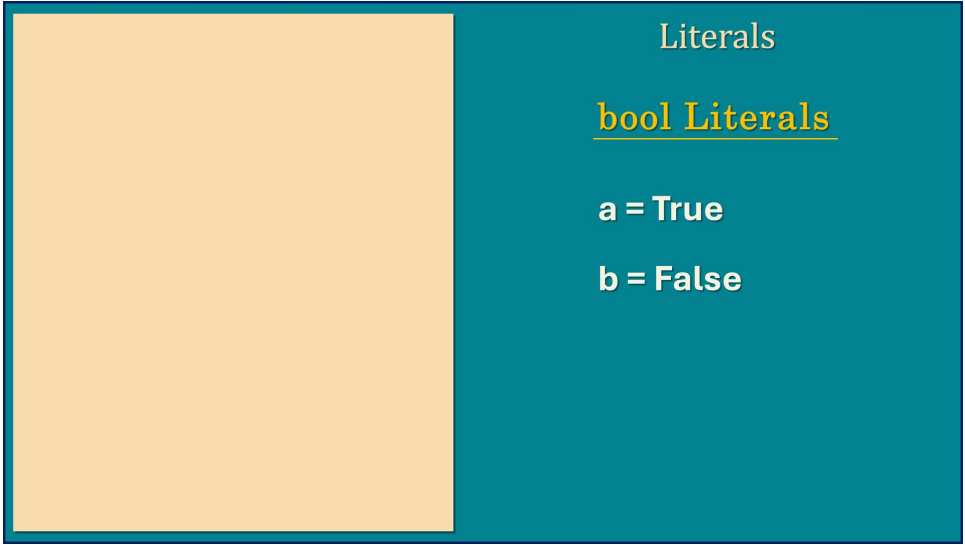
e = 12_5.6_7

f = 5._.9 ❌

Literals

bool Literals

a = True



Literals

bool Literals

a = True

b = False

Literals

bool Literals

a = True

b = False

c = false

Literals

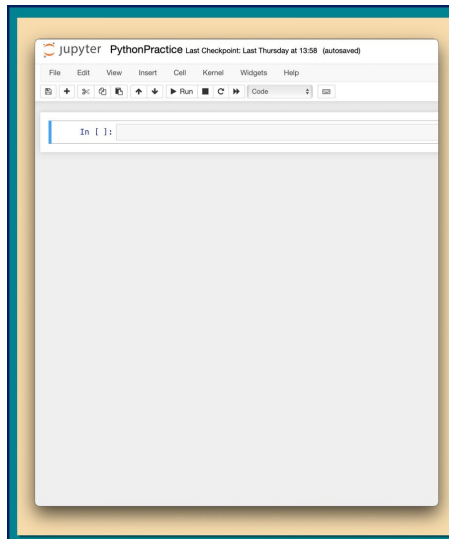
bool Literals

a = True

b = False

c = false ❌

d = 1



Literals

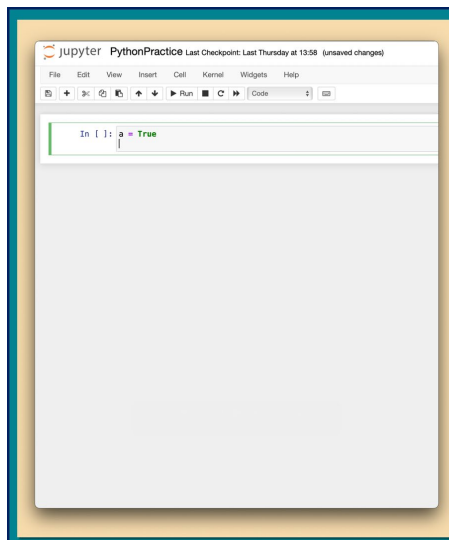
bool Literals

a = True

b = False

c = false ❌

d = 1 ❌



Literals

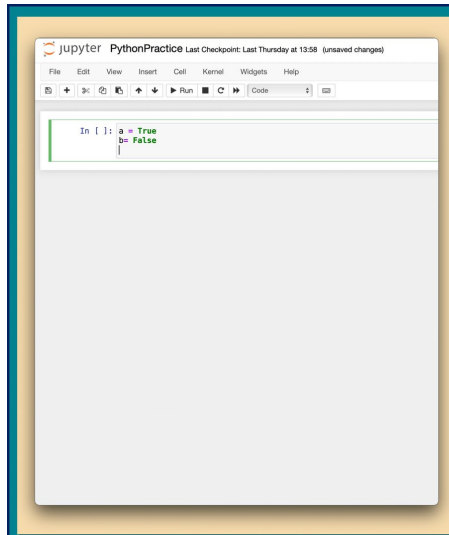
bool Literals

a = True

b = False

c = false ❌

d = 1 ❌



```
In [ ]: a = True
        b = False
```

Literals

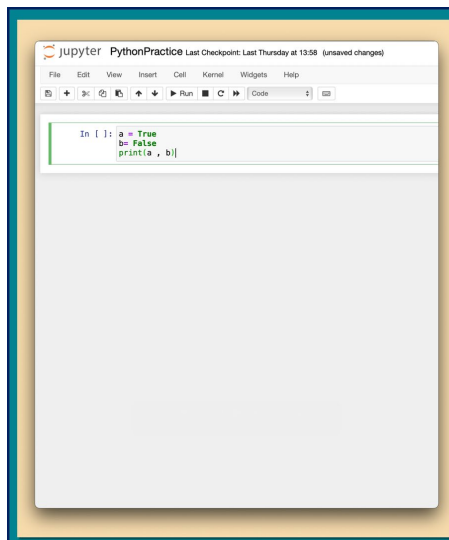
bool Literals

a = True

b = False

c = false ❌

d = 1 ❌



```
In [ ]: a = True
        b = False
        print(a, b)
```

Literals

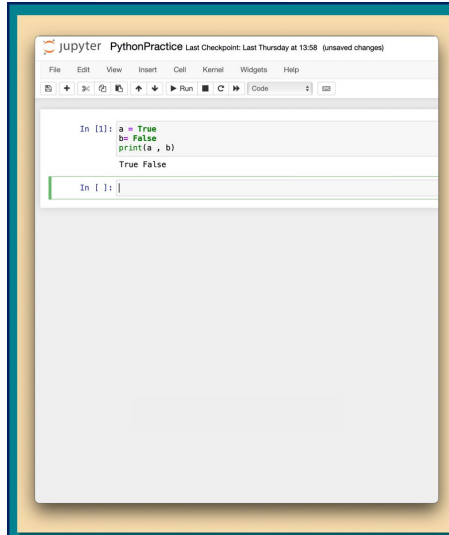
bool Literals

a = True

b = False

c = false ❌

d = 1 ❌



```
jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
+ + + + + Run + + + + + Code + + + + +
In [1]: a = True
        b = False
        print(a, b)
        True False
In [ ]:
```

Literals

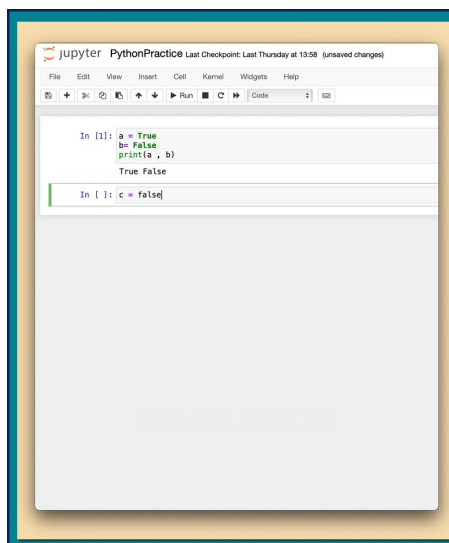
bool Literals

a = True

b = False

c = false ❌

d = 1 ❌



```
jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
+ + + + + Run + + + + + Code + + + + +
In [1]: a = True
        b = False
        print(a, b)
        True False
In [ ]: c = false
```

Literals

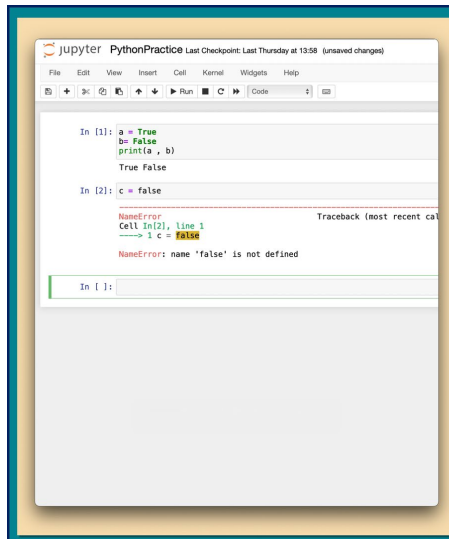
bool Literals

a = True

b = False

c = false ❌

d = 1 ❌



A Jupyter Notebook interface titled 'PythonPractice' with a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar. The notebook contains two input cells. The first cell, 'In [1]:', contains the code `a = True`, `b = False`, `print(a, b)`, and displays the output `True False`. The second cell, 'In [2]:', contains the code `c = false` and displays a `NameError` traceback: `NameError: name 'false' is not defined`. The error message indicates the issue is on line 1 of cell In[2].

Literals

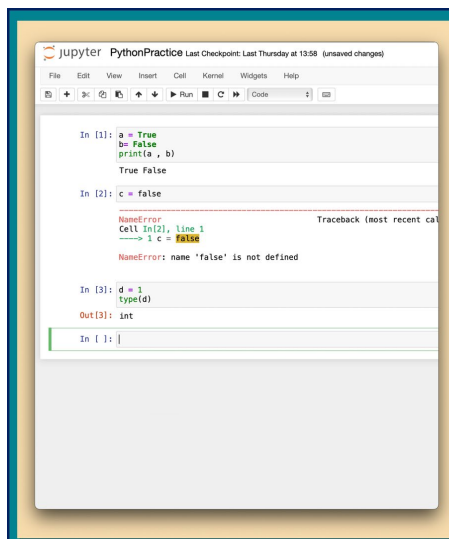
bool Literals

a = True

b = False

c = false ❌

d = 1 ❌



A Jupyter Notebook interface titled 'PythonPractice' with a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar. The notebook contains three input cells. The first cell, 'In [1]:', contains the code `a = True`, `b = False`, `print(a, b)`, and displays the output `True False`. The second cell, 'In [2]:', contains the code `c = false` and displays a `NameError` traceback: `NameError: name 'false' is not defined`. The error message indicates the issue is on line 1 of cell In[2]. The third cell, 'In [3]:', contains the code `d = 1` and `type(d)`, displaying the output `int`.

Literals

bool Literals

a = True

b = False

c = false ❌

d = 1 ❌

Literals

complex Literals

Literals

complex Literals

$$a = 4 + 5j$$

Literals

complex Literals

$$a = 4 + 5j$$

$$b = 12 + 14j$$



Literals

complex Literals

$a = 4 + 5j$

$b = 1_2 + 14j$



Literals

complex Literals

$a = 4 + 5j$

$b = 1_2 + 1_4j$

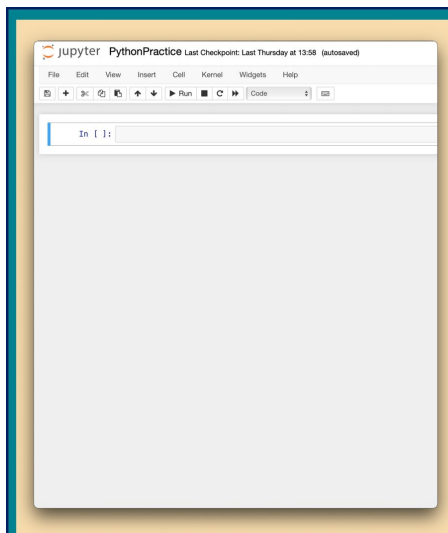
Literals

complex Literals

$a = 4 + 5j$

$b = 1_2 + 1_4j$

$c = 1.4 + 2.5j$



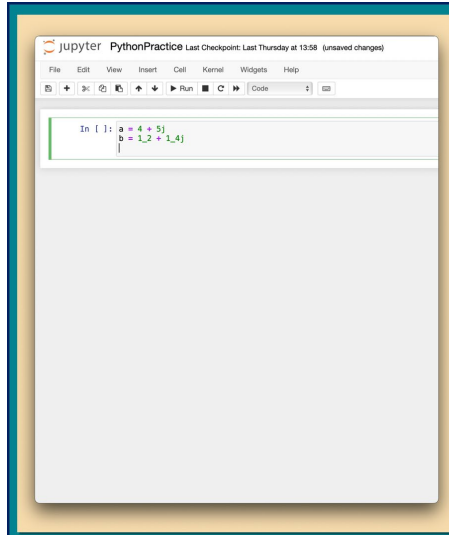
Literals

complex Literals

$a = 4 + 5j$

$b = 1_2 + 1_4j$

$c = 1.4 + 2.5j$



A screenshot of a Jupyter Notebook interface. The title bar says 'jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar has icons for new, open, save, run, and other actions. The code editor shows the following Python code:

```
In [ ]: a = 4 + 5j  
        b = 1_2 + 1_4j
```

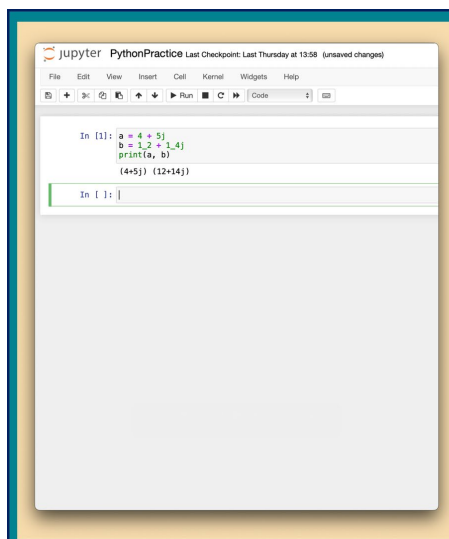
Literals

complex Literals

$$a = 4 + 5j$$

$$b = 1_2 + 1_4j$$

$$c = 1.4 + 2.5j$$



A screenshot of a Jupyter Notebook interface, similar to the one above. The code editor shows the following Python code:

```
In [1]: a = 4 + 5j  
        b = 1_2 + 1_4j  
        print(a, b)  
        (4+5j) (12+14j)  
  
In [ ]: 
```

Literals

complex Literals

$$a = 4 + 5j$$

$$b = 1_2 + 1_4j$$

$$c = 1.4 + 2.5j$$

Literals
string Literals

Literals
string Literals
name = 'Alexa'

Literals

string Literals

name = 'Alexa'

name = "Alexa"

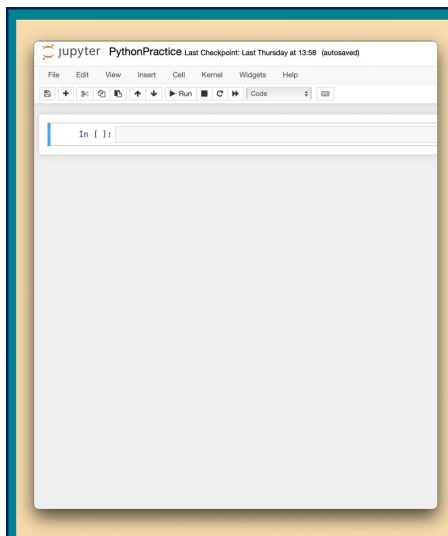
Literals

string Literals

`name = 'Alexa'`

`name = "Alexa"`

`name = """Alexa"""`



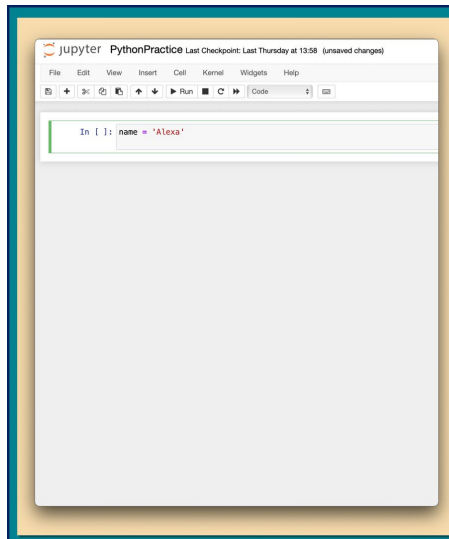
Literals

string Literals

`name = 'Alexa'`

`name = "Alexa"`

`name = """Alexa"""`

A screenshot of a Jupyter Notebook interface. The title bar shows 'jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar contains icons for new, open, save, undo, redo, run, and code. The code editor shows a single line of Python code: `In [ ]: name = 'Alexa'`.

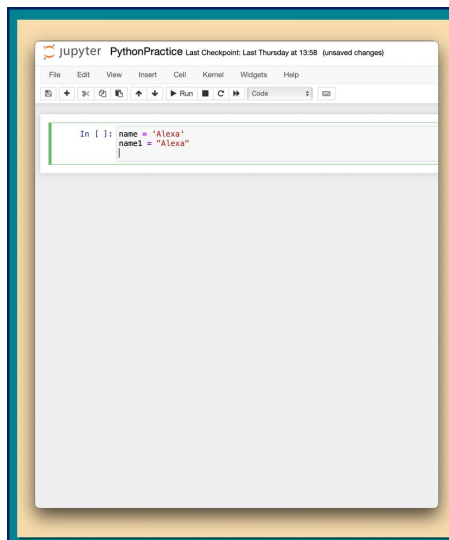
Literals

string Literals

`name = 'Alexa'`

`name = "Alexa"`

`name = """Alexa"""`

A screenshot of a Jupyter Notebook interface, similar to the one above. The code editor shows two lines of Python code: `In [ ]: name = 'Alexa'` followed by `name1 = "Alexa"` on the next line.

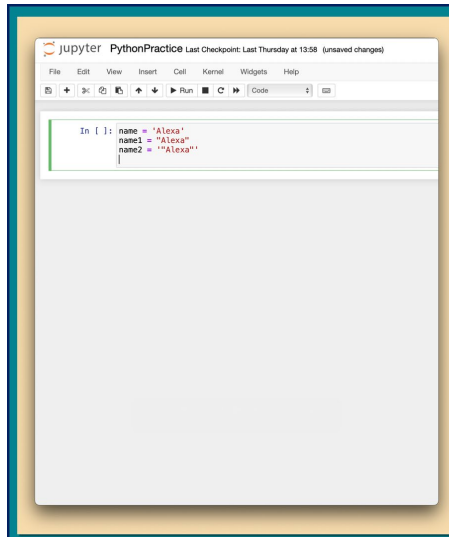
Literals

string Literals

`name = 'Alexa'`

`name = "Alexa"`

`name = """Alexa"""`



```
In [ ]: name = 'Alexa'
        name1 = 'Alexa'
        name2 = 'Alexa'
```

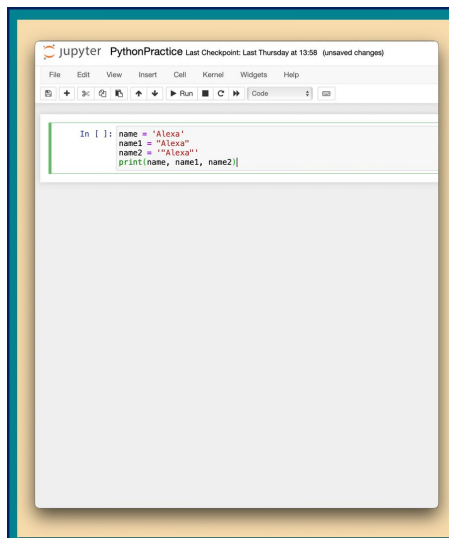
Literals

string Literals

`name = 'Alexa'`

`name = "Alexa"`

`name = """Alexa"""`



```
In [ ]: name = 'Alexa'
        name1 = 'Alexa'
        name2 = 'Alexa'
        print(name, name1, name2)
```

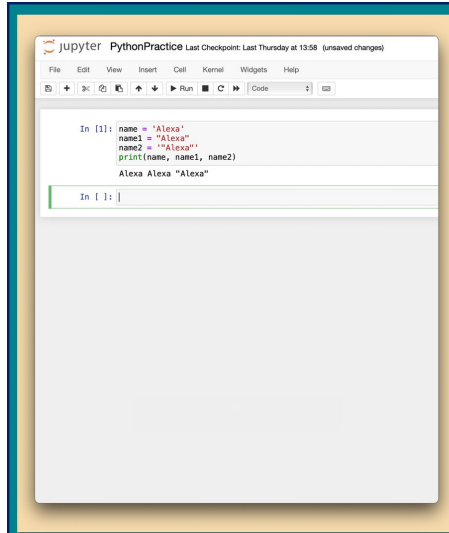
Literals

string Literals

`name = 'Alexa'`

`name = "Alexa"`

`name = """Alexa"""`

A screenshot of a Jupyter Notebook interface. The title bar shows 'jupyter PythonPractice Last checkpoint: Last Thursday at 13:58 (unsaved changes)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar contains icons for running, saving, and other actions. The code area shows a single cell with the following Python code:

```
In [1]: name = 'Alexa'
        name1 = "Alexa"
        name2 = ""Alexa""
        print(name, name1, name2)

        Alexa Alexa "Alexa"
```

The output area below the code is empty.

Literals

string Literals

name = 'Alexa'

name = "Alexa"

name = ""Alexa""